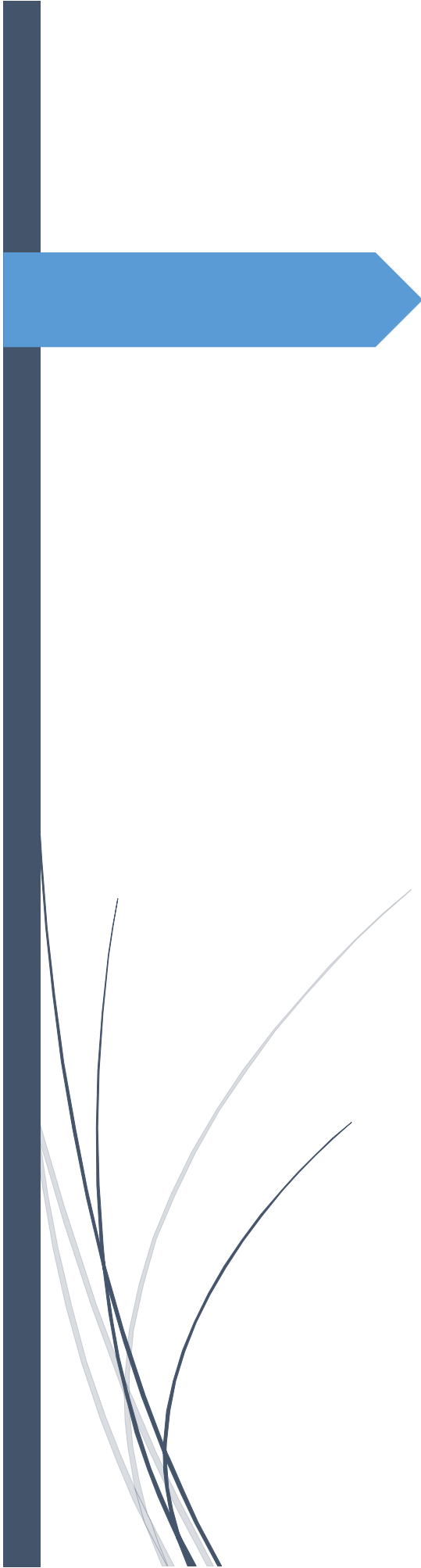




AI project

"N-Queen"

Google Drive URL:

https://drive.google.com/drive/folders/1o29EpgfdLM6WWyh-gIRilcSRWiyeMYin?usp=drive_link

Name	ID	level	Department
محمد عبد المنعم جمال الدين	931240312	2	DS
يوسف اشرف عبد المجيد محمود	933240175	2	Multimedia
محمد ياسر عبد الموجود عبد المريد	933240131	2	DS
فارس محمد سيد	931240247	2	DS
مروان اشرف نبيل ابراهيم	933240134	2	DS
محمد أسامة محمود	932240144	2	Robotics
محمد مصطفى احمد محمد	931240319	2	DS
محمد عاطف محمود المهدي	931240311	2	DS

Comparative Solving of the N-Queens Problem Using Multiple Search Algorithms

Main Idea:

The N-Queens problem asks to place N chess queens on an $N \times N$ board so that no two queens attack each other (same row, column, or diagonal). Your project implements four different algorithms to solve this problem for a user-specified N:

- Backtracking – exact, systematic DFS with pruning.
- Hill Climbing – local search that moves to the best neighbouring state.
- Best-First Search – uses a priority queue (heuristic = number of conflicts) to explore states.
- Genetic Algorithm – evolves a population of candidate boards using selection, crossover, and mutation.

A graphical user interface (GUI) lets the user enter N, runs all four algorithms sequentially, and displays each solution's board layout as well as performance metrics (execution time, number of steps, success/failure). Finally, a comparison table is shown. The project aims to demonstrate and compare the behaviour of exact, local, and metaheuristic search methods on a classic combinatorial optimisation problem.

Main Functionalities

1. User inputs board size (N) – via an entry field in the GUI.

2. Solve using four algorithms (one after another):

- Backtracking (deterministic, complete)
- Hill Climbing (stochastic, incomplete but fast)
- Best-First Search (heuristic-guided)
- Genetic Algorithm (population-based)

3. Display each solution board – as a text grid where Q represents a queen.

4. how performance metrics per algorithm:

- Execution time (seconds)
- Number of search steps / generations
- Success (whether a solution was found)

5. Generate a comparison table – summarising time, steps, and success for all four algorithms in one view.

6. Multithreaded execution – the GUI stays responsive while algorithms are running.

7. Utility functions – random board generation and conflict/cost calculation (shared by hill climbing, best-first, and genetic).

Main File

```
from display import run_gui
```

```
if __name__ == "__main__":  
    run_gui()
```

→ it calls the GUI Function to start all the operations that in the display file which display the whole project from the user input to the final displayed output.

Backtracking Search Algorithm

The Main idea of Backtracking search algorithm:

Backtracking search is a general, systematic algorithm for solving constraint satisfaction and combinatorial search problems. Its core idea is to build a solution incrementally, one step at a time, and abandon (“backtrack”) a partial candidate as soon as it is determined that the candidate cannot possibly lead to a valid complete solution.

How it works:

It explores the state space using a depth-first traversal of an implicit search tree.

At each step, it selects an unassigned variable, tries a possible value from its domain, and checks whether the assignment violates any constraints.

If the assignment is consistent, it recurses to the next variable.

If a dead end is reached (no value can be assigned without violating constraints), it backtracks – undoes the last assignment and tries the next value.

This process continues until a complete valid assignment is found or all possibilities are exhausted (proving no solution exists).

features:

- Incremental construction – decisions are made one at a time, never guessing the whole solution at once.

Pruning – large parts of the search space are eliminated early by stopping at the first inconsistency.

Completeness – if a solution exists, backtracking will eventually find it (assuming finite domains); it can also be adapted to find all solutions or optimal ones.

Memory efficiency – only the current path is stored ($O(\text{depth})$ space), unlike some algorithms that keep the whole frontier.

The Implementation of N-Queen problem solver using Backtracking Search

Algorithm:

```
global N

def printSolution(board):
    for i in range(N):
        for j in range(N):
            print (board[i][j],end=' ')
        print()

def isSafe(board, row, col):
    # Check this row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    # Check lower diagonal on left side
    for i, j in zip(range(row, N, 1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    return True

def solveNQUtil(board, col):
    # base case: If all queens are placed
    # then return true

    if col >= N:
```

```

    return True

for i in range(N):
    if isSafe(board, i, col):
        # Place this queen in board[i][col]
        board[i][col] = 1

        # recur to place rest of the queens
        if solveNQUtil(board, col + 1) == True:
            return True

        # If placing queen in board[i][col]
        # doesn't lead to a solution, then
        # queen from board[i][col]
        board[i][col] = 0

# return false if the queen can not be placed in any row in this column col
return False

def solveNQ():
    board = [ [0, 0, 0, 0],
               [0, 0, 0, 0],
               [0, 0, 0, 0],
               [0, 0, 0, 0]
             ]

    if solveNQUtil(board, 0) == False:
        print ("Solution does not exist")
        return False

    printSolution(board)

    return True

solveNQ()

```

Summary of the Code Technique:

The code implements a classic **backtracking** solution for the 4 Queens problem. Its core idea: place queens column by column, and if a partial placement leads to a dead end, undo the last decision and try another row.

Technique in the code:

- The board is a 4x4 matrix of 0s (empty) and 1s (queen).
- solveNQUtil(board, col) tries to place a queen in the current col.
 - Base case: $col == N \rightarrow$ all queens placed successfully, return True.
 - For each row "i" in that column:
 1. Use isSafe(board, i, col) to check for conflicts "only on the left side (row, upper left diagonal, lower left diagonal) because columns to the right are still empty.
 2. If safe, set $board[i][col] = 1$ and recursively call solveNQUtil(board, col+1).
 3. If the recursive call returns True, propagate the success.
 4. Otherwise, backtrack: reset $board[i][col] = 0$ and try the next row.
 - If no row works in this column, return False (triggering backtracking in the caller).
- The driver solveNQ() initializes the board and calls solveNQUtil(board, 0), then prints the first valid solution found.

Thus, the code demonstrates depth first search with immediate constraint checking and backtracking to systematically explore the search space until a solution is found.

Analysis of the code:

Time Complexity:

The algorithm explores possible queen placements column by column. In the worst case, it essentially generates all permutations of row assignments. There are at most $n!$ placements to consider, because each column can choose from n rows and choices shrink as queens are placed—but due to safety checks, the branching factor decreases. For each placement, the isSafe() function scans the left side in $O(n)$ time. Hence the overall worst-case time is $O(n! * n)$.

Space Complexity:

- The chessboard is stored as an $n*n$ matrix $\rightarrow O(n^2)$ space.

- The recursion stack goes at most depth $n \rightarrow O(n)$.
- No extra dynamic structures are used.

So the total space complexity is $O(n^2)$.

The Implemented Code of N-Queen problem solver using Backtracking Search Algorithm "Optimized":

```
import time

def solve(n):

    start_time = time.time()

    steps = 0

    #Board Index = ROW (x-axis)

    #board[x] = y (x = row, y = column)

    board = [-1] * n # board[row] = column of queen in that row

    def is_safe(x, y): # x = proposed row, y = proposed column

        for r in range(x): # Check all previously placed rows (0 to x-1)

            c = board[r] # Column of queen in row 'r'

            #check if same column

            if c == y:

                return False

            #check if same diagonal

            # Distance in X (rows) must equal Distance in Y (columns)

            if abs(r - x) == abs(c - y):

                return False

        return True

    def backtrack(x): #x = current row we are trying to fill

        nonlocal steps

        steps += 1
```

```

#base case: all rows filled
if x == n:
    return True

#try placing queen in each column of the current row
for y in range(n):
    if is_safe(x, y):
        board[x] = y # Place queen at (x, y)
        if backtrack(x + 1):
            return True
        board[x] = -1 #undo
    return False

success = backtrack(0)
end_time = time.time()
metrics = {
    "algorithm": "Backtracking",
    "time": end_time - start_time,
    "steps": steps,
    "success": success
}

# Return format: board[x] = y
return board if success else None, metrics

```

Summary of its Technique:

The code solves the N-Queens problem using backtracking with a compact 1D board representation. The idea is the same: place queens row by row, check for conflicts with already placed queens, and if a partial assignment cannot lead to a solution, undo the last placement and try the next column.

Technique in the code:

Board representation: board is a list of length n , where board[row] stores the column index of the queen in that row (or -1 if unassigned). This implicitly enforces one queen per row.

Safety check (is_safe(x,y)): For a proposed queen at row x and column y , it inspects all rows r from 0 to $x-1$ (previously placed queens). It rejects if any existing queen shares the same column (board[r] == y) or the same diagonal (abs(r - x) == abs(board[r] - y)). No need to check rows or columns ahead because they are still empty.

Recursive search (backtrack(x)):

Base case: $x == n$ means all rows have queens $\rightarrow$ solution found, return True.

For the current row x , it tries every column y in $0..n-1$:

If is_safe, place the queen (board[x] = y), then recursively attempt the next row (backtrack(x+1)).

If the recursive call succeeds, propagate True upward.

Otherwise, backtrack by resetting board[x] = -1 and try the next column.

If no column works in this row, return False to trigger backtracking in the previous row.

The driver solve(n) records the initial time and a step counter (incremented on each backtrack call), runs the search, and returns the solution (or None) together with metrics (time, steps, success).

This formulation explores the implicit search tree depth-first, prunes invalid branches early, and uses only an $O(n)$ array for state, making it both time- and space-efficient for moderate n .

Analysis of the Code:

Time Complexity:

The algorithm searches row by row, trying all column placements for each row and backtracking on conflict. In the worst case it enumerates all permutations of column assignments (since one queen per row, all columns distinct), leading to an $O(n!)$ number of recursion nodes. Each call to backtrack invokes is_safe, which loops over the already placed rows $O(n)$. So the total time is $O(n! \times n)$. With the safety check done inline.

SO the overall worst-case time is $O(n!)$.

Space Complexity:

-The board is stored as a 1D list of length $n \rightarrow O(n)$

-The recursion depth is at most $n \rightarrow O(n)$ for the call stack.

-No auxiliary structures grow with n .

So the Total space complexity is $O(n)$.

The Differences between two Codes:

1. Board representation

- First code: 2D matrix (`board[row][col]`). Queens are stored as 1, empty as 0.
- Second code: 1D list (`board[row] = col`). Stores only the column index of the queen in each row; empty rows are -1.

2. Placement order and recursion variable

- First code: Places queens column by column; recursion parameter `col`. In each column, it tries all rows.
- Second code: Places queens row by row; recursion parameter `x` (the current row). In each row, it tries all columns.
- This effectively swaps the role of rows and columns between the two versions.

3. Safety check method

- First code: `isSafe(board, row, col)` checks only the left side (same row left, upper-left diagonal, lower-left diagonal) because columns to the right are unfilled.
- Second code: `is_safe(x, y)` checks all previously placed rows (rows 0 to $x-1$) for column conflict and both diagonals. It does not need to scan in a separate loop per direction; the `abs(r - x) == abs(board[r] - y)` condition elegantly handles both diagonals.

4. State restoration (backtracking)

- First code: Explicitly sets `board[i][col] = 1` and later `board[i][col] = 0`.
- Second code: Assigns `board[x] = y` and later resets to `board[x] = -1` (a sentinel for unassigned). Effectively the same but with a different sentinel value.

5. Output and metrics

- First code: Just prints the board as a 2D grid; no timing or step counting.
- Second code: Returns the 1D solution list (or None) and a metrics dictionary containing elapsed time, number of backtrack calls (steps), and success flag.

6. Problem size

- First code is hardcoded for $N=4$; second is parameterised for any n .
- In Summary, both are standard backtracking; the main technical distinction lies in representation (2D vs. 1D compact) and ply orientation (column-wise vs. Row-wise construction), which leads to slightly different conflict checking loops but preserves the same underlying algorithm.

Summary of the results:

Time: Both need to explore up to $n!$ candidate placements in the worst case. Each `isSafe` scans previously placed queens, costing $O(n)$

$O(n)$ time (left side rows/columns for 2D, all prior rows for 1D). Hence the dominant term is $O(n! \times n)$.

The second code's safety check is asymptotically identical; sometimes it's loosely given as $O(n!)$ when the linear factor is considered negligible, but strictly it's $O(n! \times n)$.

Space: The first code stores the whole $N \times N$ board, using $O(n^2)$ memory.

The second code stores only a 1D list of column positions, using $O(n)$ memory.

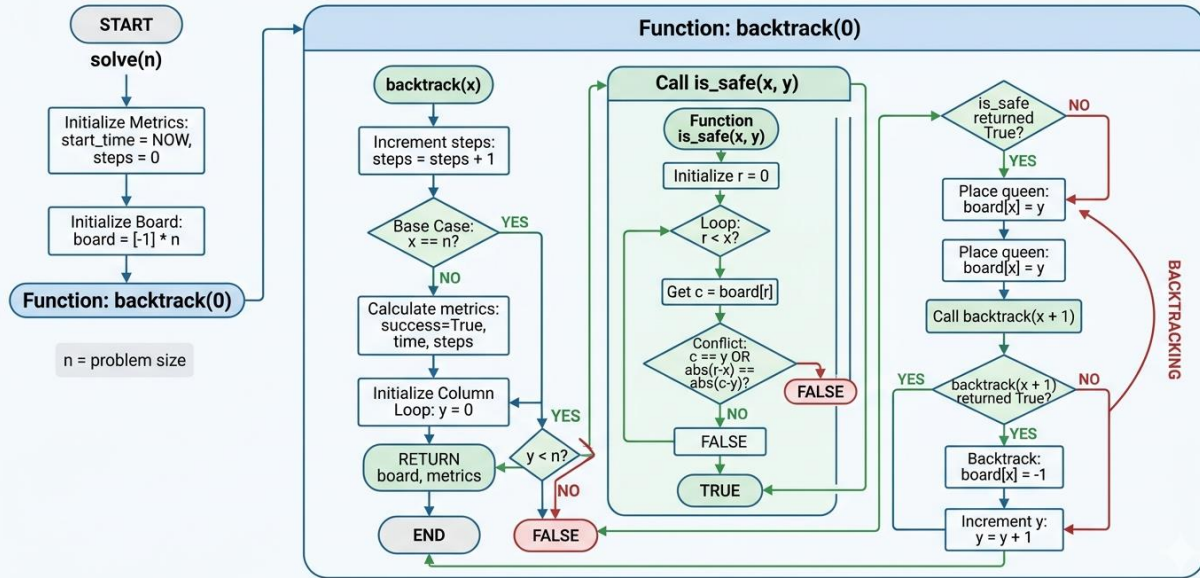
Both also use $O(n)$ recursion stack space, which is included in the total.

Comparison of Time and Space Complexity:

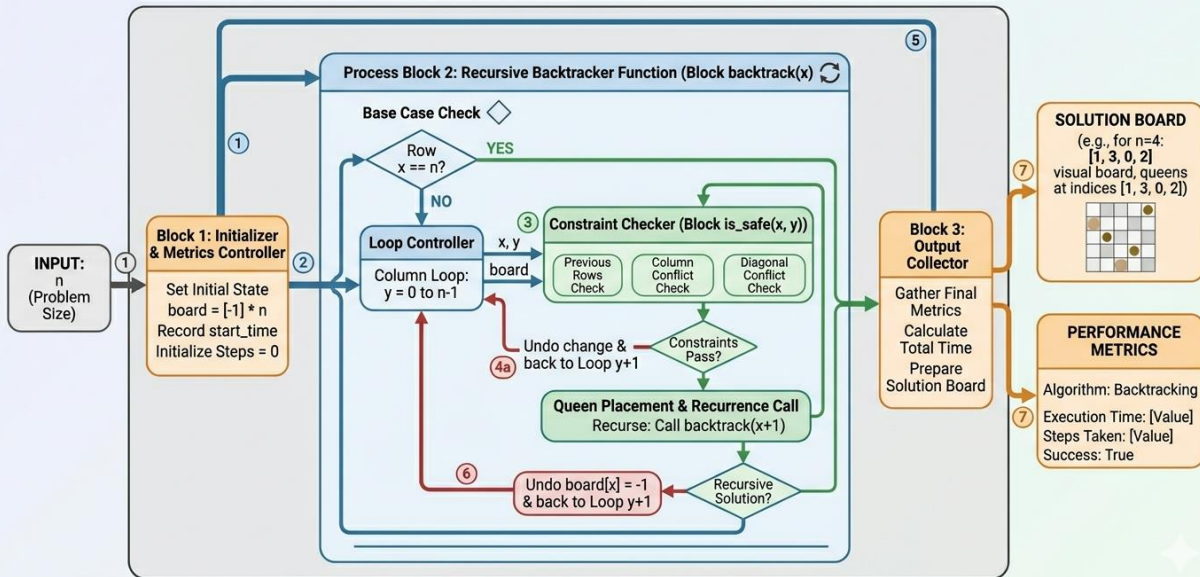
	First Code (2D)	Second Code (1D)
Time Complexity	$O(n! * n)$	$O(n! * n)$
Space Complexity	$O(n^2)$	$O(n)$
Efficiency Focus	Visual clarity, direct mapping to printed board	Memory optimization, compact data, faster state manipulation
Optimization Result	More memory, slightly slower due to 2D indexing and multiple diagonal checks	Significant memory reduction

Diagrams:

Flowchart: N-Queens Backtracking Search Algorithm



LAYERED FUNCTIONAL BLOCK DIAGRAM: N-QUEENS BACKTRACKING SEARCH ALGORITHM



Utils File

```
import random

def generate_random_board(n):
    return [random.randint(0, n - 1) for _ in range(n)]

def calculate_conflicts(board):
    n = len(board)
    conflicts = 0

    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                conflicts += 1

    return conflicts
```

→ It Contains two Essential Functions that will be used in the next 3 Algorithms (Hill Climbing Search, Best First Search and Genetic Search Algorithms), So to avoid Repeating and Code arrangement, We Implemented it in a separate file and Called it in the The other algorithms when it needed.

Note: Their analysis included in the Analysis of the optimized code for each algorithm and also their Functionalities included in the Diagrams of the optimized code for each algorithm.

Hill Climbing Search Algorithm

The Main Idea of Hill Climbing Search Algorithm:

Hill Climbing is a local search optimization algorithm used to solve problems by iteratively improving a candidate solution. Its main idea is to start with an initial complete solution (often random) and continuously move to a better neighboring solution based on an evaluation function, until no better improvement can be found.

Unlike systematic search methods, Hill Climbing does not explore the entire search space. Instead, it focuses only on local improvements, making it a greedy approach that always chooses the best immediate option.

How it works:

- It begins with a randomly generated complete state (initial solution).
- At each step, the algorithm evaluates the current state using an objective (cost) - function that measures how good the solution is.
- It then generates all possible neighboring states by making small modifications to the current solution (e.g., moving one queen to another position in the same column).
- Among all neighbors, it selects the one with the best (lowest or highest depending on the problem) evaluation value.
- If the selected neighbor is better than the current state, the algorithm moves to it.
- If no better neighbor exists, the algorithm stops, since it has reached a local optimum (or uses random restart in some variations to escape it).
- This process repeats until a solution with optimal or acceptable quality is found.

Features:

Local search approach – it explores only neighboring states instead of the full search space.

Greedy improvement strategy – always moves toward the best immediate improvement.

No backtracking or tree expansion – it does not reconsider past decisions.

Efficient memory usage – only the current state and a few neighbors are stored ($O(n^2)$ for board representation in N-Queens).

Fast in practice but not guaranteed optimal – it may get stuck in local maxima, plateaus, or ridges.

Incomplete algorithm – it does not guarantee finding a solution even if one exists, depending on the landscape of the problem.

The Implementation of N-Queen problem solver using Hill Climbing Search Algorithm:

```
from random import randint

N = int(input("Enter number of queens: "))

def configureRandomly(board, state):

    for i in range(N):

        state[i] = randint(0, N - 1)

    generateBoard(board, state)

def printBoard(board):

    for i in range(N):

        print(*board[i])

def fill(board, value):

    for i in range(N):

        for j in range(N):

            board[i][j] = value

def generateBoard(board, state):

    fill(board, 0)

    for i in range(N):

        board[state[i]][i] = 1

def copyState(s1, s2):

    for i in range(N):

        s1[i] = s2[i]

def calculateObjective(board, state):

    attacking = 0

    for i in range(N):

        for j in range(i + 1, N):

            if state[i] == state[j]:
```

```

        attacking += 1

    elif abs(state[i] - state[j]) == abs(i - j):
        attacking += 1

    return attacking

def getNeighbour(board, state):
    bestState = [0] * N

    copyState(bestState, state)

    bestObjective = calculateObjective(board, state)

    for col in range(N):
        originalRow = state[col]

        for row in range(N):
            if row != originalRow:
                state[col] = row

                tempBoard = [[0] * N for _ in range(N)]

                generateBoard(tempBoard, state)

                tempObjective = calculateObjective(tempBoard, state)

                if tempObjective < bestObjective:
                    bestObjective = tempObjective

                    copyState(bestState, state)

        state[col] = originalRow

    copyState(state, bestState)

    generateBoard(board, state)

def hillClimbing(board, state):
    maxSteps = 1000

    steps = 0

    while steps < maxSteps:
        currentObjective = calculateObjective(board, state)

```

```

if currentObjective == 0:
    print("\nSolution Found:\n")
    printBoard(board)
    return

neighbourBoard = [[0] * N for _ in range(N)]
neighbourState = [0] * N
copyState(neighbourState, state)
generateBoard(neighbourBoard, neighbourState)
getNeighbour(neighbourBoard, neighbourState)
neighbourObjective = calculateObjective(
    neighbourBoard,
    neighbourState
)

if neighbourObjective >= currentObjective:
    configureRandomly(board, state)
else:
    copyState(state, neighbourState)
    generateBoard(board, state)

steps += 1

print("No solution found within step limit.")

state = [0] * N
board = [[0 for _ in range(N)] for _ in range(N)]
configureRandomly(board, state)
print("\nInitial Board:\n")
printBoard(board)
hillClimbing(board, state)

```

Summary of the Code Technique:

The code implements a Hill Climbing optimization algorithm to solve the N-Queens problem. Unlike backtracking, it starts with a complete random configuration and gradually improves it by moving to better neighboring states until a solution is reached or a local optimum is found.

Technique in the code:

- The board is an $N \times N$ matrix of 0s (empty) and 1s (queen), and the state array stores the row position of the queen in each column.
- The function `configureRandomly(board, state)` generates a random initial configuration by placing one queen per column in random rows.
- `CalculateObjective(board, state)` computes the number of conflicting queen pairs (attacks). This is the cost function that the algorithm tries to minimize.
- `getNeighbour(board, state)` explores all possible neighboring states by moving one queen within its column and selects the configuration with the lowest number of conflicts.
- `hillClimbing(board, state)` repeatedly:
 - 1- Generates the best neighbor using `getNeighbour`.
 - 2- Compares it with the current state.
 - 3- If the neighbor is better, it moves to it.
 - 4- If no improvement is possible, it either stops (local optimum) or performs a random restart step to escape local minima.
- The process continues until a configuration with zero conflicts is found or no better state exists.

Thus, the code demonstrates a greedy local search strategy where the algorithm continuously moves toward the best immediate improvement rather than exploring all possibilities.

Analysis of the Code

Time Complexity:

Generating the initial board: $O(N^2)$

Evaluating a state (`calculateObjective`): $O(N^2)$ due to checking multiple directions for each queen.

Generating neighbors in getNeighbour:

- For each column (N), it tries N possible row positions $\rightarrow O(N^2)$ states
- Each evaluation costs $O(N^2)$

Therefore, one full neighbor search costs approximately:
 $O(N^4)$

Hill Climbing iterations depend on convergence, so worst-case time complexity is unbounded, but practically it is much faster than brute force.

Space Complexity:

Board storage: $O(N^2)$

State array: $O(N)$

Temporary boards in neighbor generation: $O(N^2)$

No recursion is used, so no stack overhead.

- **Total Space Complexity: $O(N^2)$**

The Implemented Code of N-Queen problem solver using Hill Climbing

Search Algorithm "Optimized":

```
import time
import random
def generate_random_board(n):
    return [random.randint(0, n - 1) for _ in range(n)]
def calculate_conflicts(board):
    n = len(board)
    conflicts = 0
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                conflicts += 1
    return conflicts
def solve(n):
    start_time = time.time()
    steps = 0
    current = generate_random_board(n)
```

```

current_conflicts = calculate_conflicts(current)
best = current[:]
best_conflicts = current_conflicts
while current_conflicts > 0:
    steps += 1
    next_state = current
    next_conflicts = current_conflicts
    for row in range(n):
        for col in range(n):
            if current[row] == col:
                continue
            neighbor = current[:]
            neighbor[row] = col
            c = calculate_conflicts(neighbor)
            if c < next_conflicts:
                next_state = neighbor
                next_conflicts = c
    if next_conflicts >= current_conflicts:
        break
    current = next_state
    current_conflicts = next_conflicts
    if current_conflicts < best_conflicts:
        best = current[:]
        best_conflicts = current_conflicts
end_time = time.time()
metrics = {
    "algorithm": "Hill Climbing",
    "time": end_time - start_time,
    "steps": steps,
    "final_conflicts": current_conflicts,
    "success": current_conflicts == 0
}
return best, metrics
if __name__ == "__main__":
    n = int(input("Enter number of queens: "))
    solution, metrics = solve(n)
    print(solution)
    print(metrics)

```

Summary of its Technique :

The code solves the N-Queens problem using the Hill Climbing optimization algorithm with a compact 1-D board representation. Instead of building the solution step by step like backtracking, it starts with a complete random configuration and iteratively improves it by moving to better neighboring states until a solution is found or a local optimum is reached.

Technique in the code:

Board representation:

board is a list of length n, where board[row] stores the column index of the queen in that row. This implicitly enforces one queen per row.

Conflict calculation (calculate_conflicts):

The function counts the number of attacking pairs of queens. It checks all pairs of rows (i, j) where $i < j$ and increments conflicts if two queens share the same column ($\text{board}[i] == \text{board}[j]$) or the same diagonal ($\text{abs}(\text{board}[i] - \text{board}[j]) == \text{abs}(i - j)$).

Initial state:

The algorithm starts with a random board generated using `generate_random_board(n)`, where each row is assigned a random column.

Neighbor generation:

For each row, the algorithm tries moving the queen to every other column in the same row (except its current position), evaluating all possible neighboring states

State improvement (Hill Climbing step):

Among all generated neighbors, the algorithm selects the one with the lowest number of conflicts. If this neighbor has fewer conflicts than the current state, the algorithm moves to it.

Stopping condition:

If no neighbor improves the current state (i.e., no reduction in conflicts), the algorithm stops because it has reached a local minimum.

If the number of conflicts becomes zero, a valid solution is found.

Best state tracking:

The algorithm keeps track of the best state found during the search, even if it gets stuck before reaching a perfect solution.

The driver `solve(n)`:

It records execution time, counts the number of steps (iterations), runs the hill climbing process, and returns the best found solution along with performance metrics (time, steps, final conflicts, success).

This approach performs a greedy local search, making fast improvements but without guaranteeing reaching the global optimum due to possible local minima.

Analysis of the Code:

Time Complexity:

In each iteration, the algorithm generates all neighbors. For each of the n rows, it tries n possible columns, resulting in $O(n^2)$ neighbors.

For each neighbor, `calculate_conflicts` runs in $O(n^2)$ time (checking all pairs of queens).

So each iteration costs $O(n^4)$.

If the algorithm runs for k iterations, the total time complexity is $O(k \times n^4)$.

In practice, k is usually small, but there is no strict upper bound.

Space Complexity:

The board representation uses a list of size $n \rightarrow O(n)$

The algorithm stores only a few board configurations at a time (current, best, neighbor, and `next_state`), each requiring $O(n)$ space.

No recursion or large auxiliary data structures are used.

So the total space complexity is $O(n)$.

The Differences between two Codes :

1. Board representation

First code:

Uses a 2-D matrix (`board[row][col]`) where queens are stored as 1 and empty cells as 0. It also maintains a separate `state[]` array to track queen positions per column.

Second code (Optimized):

`board[row]` stores the column position of the queen in that row

Difference:

The first code separates board visualization and state tracking, while the second merges both into a compact representation.

2. Initial state generation

First code:

Uses `configureRandomly(board, state)` which places one queen per column in random rows while also updating a 2D board.

Second code:

Uses `generate_random_board(n)` which directly generates a 1D random configuration without constructing a board matrix.

Difference:

The second code is more lightweight and avoids unnecessary 2D structure creation.

3. Conflict / Objective function**First code:**

`calculateObjective(board, state)` checks conflicts by scanning the board in multiple directions (horizontal, diagonal, etc.).

Second code:

`calculate_conflicts(board)` checks only same column and diagonal conflicts using absolute difference.

Difference:

The second code uses a simpler and more direct mathematical conflict check, making it cleaner and faster.

4. Neighbor generation**First code:**

`getNeighbour()` explores all possible moves of each queen inside its column, rebuilds full 2D boards, and selects the best state.

Second code:

evaluates all possible neighboring states

Difference:

The second code avoids repeated full matrix reconstruction, making it more efficient.

5. Search strategy (Hill Climbing behavior)**First code:**

Repeatedly improves current state and includes random restart behavior when stuck in local optimum.

Second code:

Moves to best neighbor if better and stops immediately if no improvement is found.

Difference:

The first code is more robust and can escape local minima, while the second is strictly greedy.

6. Output and metrics

First code:

Prints the final board in 2D format only.

Second code:

Returns solution with metrics including execution time, steps, success flag, and final conflicts.

Difference:

The second code is designed for analysis and evaluation, not just solving.

7. Problem size handling

First code:

Uses global N and is less flexible.

Second code:

Fully parameterized using solve(n) and supports any input size.

Difference:

Second code is more reusable and scalable.

Summary of the results:

Time Complexity:

First code:

Each iteration costs $O(N^4)$ due to neighbor generation and objective evaluation.

Second code:

Each iteration also costs $O(N^4)$ due to generating and evaluating all neighbors.

Final:

Both algorithms have the same asymptotic complexity: $O(k \times N^4)$ where k is number of iterations.

Space Complexity:

First code:

Uses a 2D board and multiple temporary matrices $\rightarrow O(N^2)$.

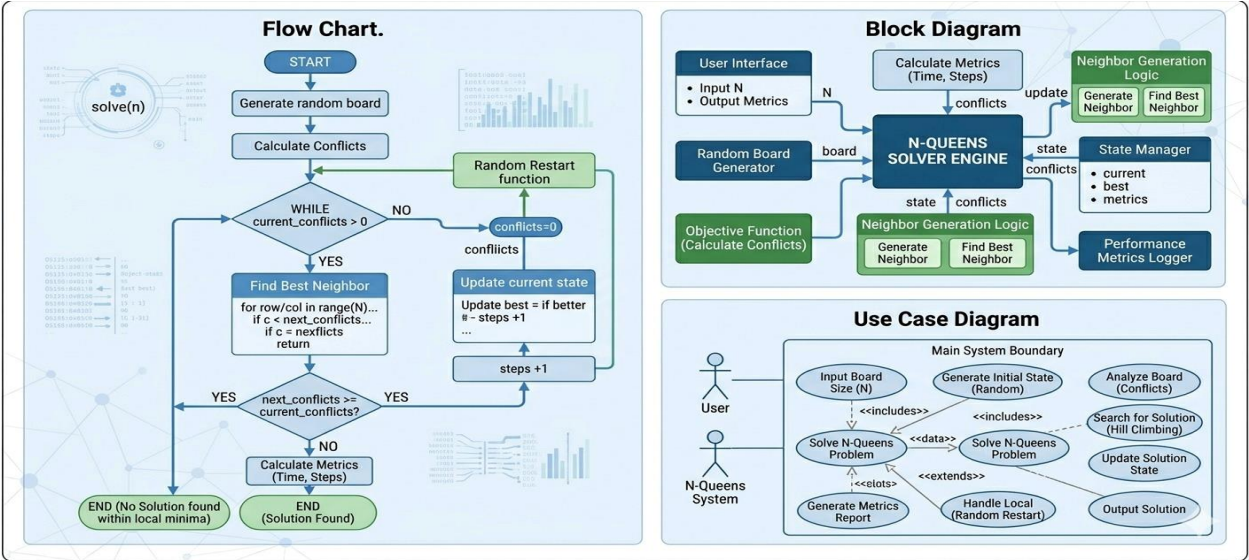
Second code:

Uses a 1D board and only a few auxiliary variables $\rightarrow O(N)$

Comparison of Time and Space Complexity:

Aspect	First Code (2D Board)	Second Code (1D Optimized)
Time Complexity (per iteration)	$O(N^4)$	$O(N^4)$
Total Time Complexity	$O(k \times N^4)$	$O(k \times N^4)$
Space Complexity	$O(N^2)$ (2D board + temporary matrices)	$O(N)$ (only state + few variables)
Auxiliary Storage	Multiple matrices and state tracking arrays	Only a few variables (current, best, temp state)
Efficiency Focus	More memory usage, clearer visualization	Memory optimized, cleaner structure
Optimization Result	Higher memory consumption	Significant space reduction

Diagrams:



Best First Search Algorithm

The Main Idea of the best First Search Algorithm:

Best-first search is a heuristic-driven graph traversal algorithm that always expands the node that appears most promising according to a given evaluation function. Instead of blindly exploring paths in a fixed order (like BFS or DFS), it prioritizes nodes with the lowest estimated cost to reach the goal, making it an informed search strategy.

How it works:

- Maintain a priority queue (often implemented as a min-heap) of discovered but unexpanded nodes.
- The evaluation function $f(n)$ determines priority. Common variants:
 - Greedy best-first search: $f(n) = h(n)$, where $h(n)$ is a heuristic estimate of cost from node n to goal.
 - A search: $f(n) = g(n) + h(n)$, where $g(n)$ is actual cost from start to n and $h(n)$ is heuristic to goal. A is optimal if h is admissible.
- At each step, dequeue the node with minimum $f(n)$, check if it's the goal, and if not, generate its successors, compute their f values, and enqueue them.

Key Characteristics:

- Explores states in order of perceived promise, often reaching solutions faster than uninformed methods.
- Memory-intensive: must store all generated nodes in the queue, potentially leading to exponential space complexity.
- Completeness and optimality depend on the heuristic and variant (e.g., greedy is incomplete with cycles; A^* is complete with a consistent heuristic).
- The quality of the heuristic critically impacts performance—poor heuristics can mislead the search.

The Implementation of N-Queen problem solver using Best first Search Algorithm:

```
from heapq import heappush, heappop
```

```
def calculate_conflicts(board, n):
```

```
    Calculate the number of attacking pairs of queens on the board
```

Lower conflict count = better board state

conflicts = 0

Check each queen's position

for i in range(n):

 if board[i] == -1: # No queen in this row

 continue

 for j in range(i + 1, n):

 if board[j] == -1: # No queen in this row

 continue

 # Same column

 if board[i] == board[j]:

 conflicts += 1

 # Same diagonal (difference in rows = difference in columns)

 if abs(board[i] - board[j]) == abs(i - j):

 conflicts += 1

return conflicts

def get_possible_moves(board, n):

 Generate all possible next board states by placing a queen in the next empty row

 Returns list of (conflicts, board_state, row, col) for each possible move

 # Find first empty row (row with no queen)

 next_row = -1

 for i in range(n):

 if board[i] == -1:

 next_row = i

 break

 if next_row == -1: # Board is full

 return []

```

possible_moves = []

# Try placing queen in each column of the next row
for col in range(n):

    new_board = board.copy()

    new_board[next_row] = col

    conflicts = calculate_conflicts(new_board, n)

    possible_moves.append([conflicts, new_board, next_row, col])

return possible_moves

def bestFirstNQueen(n):

    # Create initial board (all rows empty, represented by -1)
    initial_board = [-1] * n

    # Priority queue (min heap) to store board states by their conflict count
    pq = []

    # Push the initial state
    # Format: [conflicts, board_state, current_row, last_placed_col]
    heappush(pq, [0, initial_board, -1, -1])

    # To keep track of visited states (to avoid cycles)
    # We'll store a tuple of the board as the visited key
    visited = set()
    visited.add(tuple(initial_board))

    # To store the path of moves (optional)
    path = []

    # Counter for explored states
    states_explored = 0

    print(f"Starting Best First Search for {n}-Queen problem...")
    print("=" * 50)

    while pq:

```

```

# Get the board state with the least conflicts
current_conflicts, current_board, last_row, last_col = heappop(pq)

states_explored += 1

# Store in path (for demonstration)
if last_row != -1:
    path.append((last_row, last_col))

# Print current best state (optional visualization)
if states_explored % 1000 == 0 or current_conflicts == 0:
    print(f"Step {states_explored}: Conflicts = {current_conflicts}")
    if current_conflicts < 10: # Only print small conflict states
        print_board(current_board, n)
        print()

# Check if we found a solution (no conflicts)
if current_conflicts == 0 and -1 not in current_board:
    print(f"\n✓ SOLUTION FOUND after exploring {states_explored} states!")
    return current_board, path, states_explored

# Generate all possible next moves
possible_moves = get_possible_moves(current_board, n)

# Add each valid move to the priority queue
for conflicts, new_board, row, col in possible_moves:
    board_tuple = tuple(new_board)

    # Only add if we haven't visited this state before
    if board_tuple not in visited:
        visited.add(board_tuple)
        heappush(pq, [conflicts, new_board, row, col])

print(f"\nX No solution found for {n}-Queen after exploring {states_explored} states!")

return None, path, states_explored

```

```

def print_board(board, n):

    print(" " + " ".join([str(i) for i in range(n)]))

    for i in range(n):

        row_str = f"{i} "

        for j in range(n):

            if board[i] == j:

                row_str += "Q "

            else:

                row_str += ". "

        print(row_str)

def print_solution_summary(board, n, states_explored):

    if board is None:

        print("No solution exists!")

        return

    print("\n" + "=" * 50)

    print(f"SOLUTION FOR {n}-QUEEN PROBLEM")

    print("=" * 50)

    print_board(board, n)

    print("\nQueen positions (row: column):")

    for row in range(n):

        print(f" Row {row} -> Column {board[row]}")

    # Verify solution

    verify_solution(board, n)

    print(f"\nTotal states explored: {states_explored}")

    print(f"Best First Search completed!")

def verify_solution(board, n):

    conflicts = calculate_conflicts(board, n)

```

```

if conflicts == 0 and -1 not in board:

    print(f"✓ VERIFIED: No conflicts found! Valid solution.")

else:

    print(f"X ERROR: Found {conflicts} conflicts! Invalid solution.")

def solve_multiple_queens():

    print("\n" + "=" * 60)

    print("N-QUEEN PROBLEM SOLVER - BEST FIRST SEARCH")

    print("=" * 60)

    # Test different N values

    test_sizes = [4, 8, 10]

    for n in test_sizes:

        print(f"\n{'=' * 60}")

        print(f"Solving {n}-Queen Problem")

        print(f"{'=' * 60}")

        solution, path, states = bestFirstNQueen(n)

        print_solution_summary(solution, n, states)

        if solution:

            # Show the move order (first few moves)

            print(f"\nFirst 10 moves (row, col) in search path:")

            for i, (row, col) in enumerate(path[:10]):

                print(f" Move {i + 1}: Place queen at ({row}, {col})")

            if len(path) > 10:

                print(f" ... and {len(path) - 10} more moves")

def solve_custom_queen():

    print("\n" + "=" * 60)

    print("CUSTOM N-QUEEN PROBLEM")

    print("=" * 60)

```

```

try:

    n = int(input("Enter the board size (N) for N-Queen problem: "))

    if n < 1:

        print("Please enter a positive integer!")

        return

    if n > 12:

        print(f"Warning: {n}-Queen may take a very long time to solve!")

        response = input("Continue anyway? (y/n): ")

        if response.lower() != 'y':

            return

    print(f"\nSolving {n}-Queen problem...\n")

    solution, path, states = bestFirstNQueen(n)

    print_solution_summary(solution, n, states)

except ValueError:

    print("Please enter a valid integer!")

if __name__ == "__main__":

    # Option 1: Solve for multiple board sizes automatically

    solve_multiple_queens()

    # Option 2: Uncomment below to solve for custom board size

    # solve_custom_queen()

    # Option 3: Solve for a specific N (like the original code's example)

    print("\n" + "=" * 60)

    print("EXAMPLE: Solving 8-Queen Problem (Classic)")

    print("=" * 60)

    n = 8

    print(f"\nSolving {n}-Queen problem...\n")

    solution, path, states = bestFirstNQueen(n)

```

if solution:

```
print("\n✓ SUCCESS! Final solution for 8-Queen:")
```

```
print_board(solution, n)
```

```
# Show queen positions in a list format (similar to the original code's output)
```

```
print("\nQueen positions array (index=row, value=column):")
```

```
print(solution)
```

```
# Verify all rows have queens
```

if -1 not in solution:

```
print("\n✓ All rows have a queen placed!")
```

```
total_conflicts = calculate_conflicts(solution, n)
```

```
print(f"Total conflicts in solution: {total_conflicts}")
```

Summary of the code technique:

Idea:

The code solves the N-Queens problem using backtracking, a depth-first search that builds a solution incrementally and abandons partial solutions (backtracks) as soon as they violate constraints. It places queens column by column, ensuring no two queens threaten each other.

Technique:

1. Board Representation: A 2D list board of size $N \times N$ initialized with "0". A "1" marks a placed queen.

2. Recursive Placement (solveNQUtil):

- Parameters: board and current col (column index, 0 to $N-1$).
- Base Case: If $col \geq N$, all queens are placed; return True.
- Column Loop: Try placing a queen in each row "i" of col.
 - Call "isSafe(board, i, col)" to check if position (i, col) is threatened from the left side only (since columns to the right are empty yet).
 - Safety Check Details: Scans the row on the left, then the upper-left diagonal, then the lower-left diagonal for any existing queen (` 1 `). If none found, position is safe.
 - If safe, set "board[i][col] = 1".
- Recurse to the next column: "if solveNQUtil(board, col+1) == True: return True". This propagates success back up the recursion.
- Backtrack: If recursion returns "False" (no valid placement in subsequent columns), reset "board[i][col] = 0" to remove the queen and try the next row.
- If no row in col works, return False to trigger backtracking in the previous column.

3. Driver (solveNQ):

Creates the empty board of size 4, calls "solveNQUtil(board, 0)". If a solution exists, it prints the board showing queen positions.

Analysis of the code:

Time Complexity:

Let V = number of nodes (n) and E = number of edges.

Building the adjacency list takes $O(E)$.

Each node is pushed into the min-heap at most once (because visited is set to True before pushing), and popped once $\rightarrow O(V \log V)$ for heap operations.

For every node popped, we iterate over all its neighbors. Summed over all nodes, that's $O(E)$ iterations. Inside each iteration, a push may occur (another $O(\log V)$).

Total time = $O((V + E) \log V)$, which simplifies to $O(E \log V)$ for connected graphs.

In the worst case (dense graph), $E \approx V^2$, giving $O(V^2 \log V)$.

Space Complexity:

Adjacency list: $O(V + E)$ to store all nodes and edges.

visited array: $O(V)$.

Min-heap pq: at most $O(V)$ entries.

path list: $O(V)$.

Total space = $O(V + E)$ (dominated by the adjacency list). For a dense graph, this becomes $O(V^2)$

The Implementation of N-Queen problem solver using Best first Search Algorithm "Optimized":

```
import time
```

```
import heapq
```

```
from utils import generate_random_board, calculate_conflicts
```

```
def solve(n):
```

```
    start_time = time.time()
```

```
    steps = 0
```

```
    #initial state
```

```
initial = generate_random_board(n)

#priority queue (min-heap)

pq = []

heapq.heappush(pq, (calculate_conflicts(initial), initial))

visited = set()

best_solution = None

#search loop

while pq:

    steps += 1

    conflicts, state = heapq.heappop(pq)

    state_tuple = tuple(state)

    if state_tuple in visited:

        continue

    visited.add(state_tuple)

    #goal state

    if conflicts == 0:

        best_solution = state

        break

    #generate neighbors

    for row in range(n):

        for col in range(n):

            if state[row] == col:

                continue

            neighbor = state[:]

            neighbor[row] = col

            heapq.heappush(

                pq,
```

```

        (calculate_conflicts(neighbor), neighbor)
    )
end_time = time.time()

metrics = {
    "algorithm": "Best First Search",
    "time": end_time - start_time,
    "steps": steps,
    "success": best_solution is not None
}

return best_solution, metrics

```

Summary of the Code Technique:

The code performs a greedy best-first search to solve the N-Queens problem. It always expands the board configuration that has the fewest queen conflicts (heuristic), believing that moving toward fewer conflicts will lead fastest to a solution.

Technique:

Initial State: A random placement of one queen per row is generated.

Priority Queue (Min-Heap): The queue stores (conflict_count, board_state). The state with the smallest conflict count is popped first.

Goal Check: If a popped state has conflicts == 0, it is a solution and the search stops.

Visited Set: A set of tuple(state) prevents re-exploring identical configurations.

Neighbor Generation: For the current state, every possible move of a single queen in each row to a different column is generated. This produces $(n-1) * n$ neighbors per state.

Evaluation: Each neighbor is evaluated by calculate_conflicts (counting attacking queen pairs) and pushed into the heap.

Analysis of the code:

Time Complexity (worst-case):

Per expanded state: $O(n^3)$ (efficient conflict check) or $O(n^4)$ (naive check).

Maximum expanded states: $O(n^n)$ (all possible row-wise assignments).

Total: $O(n^{n+3})$ (efficient) or $O(n^{n+4})$ (naive).

Space Complexity (worst-case):

Visited set: $O(n \cdot n^n)$.

Priority queue: $O(n^3 \cdot n^n)$ (duplicate neighbours accumulated).

Overall: $O(n^{n+3})$.

The Differences between two Codes :

1. Problem Domain

Earlier code (Graph BFS): Pathfinding in a weighted undirected graph, searching for a route from a source node to a target node.

Previous code (N-Queens BFS): Constraint satisfaction, searching for a configuration of N queens with zero attacks.

2. Heuristic / Evaluation Function

Graph BFS: Uses edge cost as priority (greedy by local edge weight). It does not incorporate a heuristic estimate to the goal — it's essentially a uniform-cost variant or greedy on immediate step cost.

N-Queens BFS: Uses conflict count (number of attacking queen pairs) as the heuristic $h(n)$. This directly estimates "distance to goal" and guides the search toward low-conflict states.

3. State Representation

Graph BFS: State is a single integer (node ID).

N-Queens BFS: State is a list/array of length n , representing column positions of each queen.

4. Neighbour Generation

Graph BFS: Neighbors are fixed per node, read from an adjacency list. Only unvisited neighbours are pushed.

N-Queens BFS: Neighbours are generated dynamically by perturbing one queen per row to every other column. This creates $O(n^2)$ neighbours per state. Duplicate states may be generated and pushed before visitation is checked.

5. Visited Mechanism

Graph BFS: Boolean visited array, marked before pushing into the heap (avoids duplicate pushes).

N-Queens BFS: visited set of state tuples, checked after popping. Duplicates can accumulate in the heap and are discarded when popped.

6. Goal Condition

Graph BFS: `current_node == target`.

N-Queens BFS: `conflicts == 0` (heuristic value zero).

7. Algorithmic Strategy

Graph BFS expands nodes based on the smallest single-step cost, behaving like a greedy local search without lookahead.

N-Queens BFS expands states minimal conflicts, a pure best-first hill-climbing variant that always moves toward the most promising (lowest conflict) neighbour. It does not account for path length; it seeks any solution.

8. Completeness & Optimality

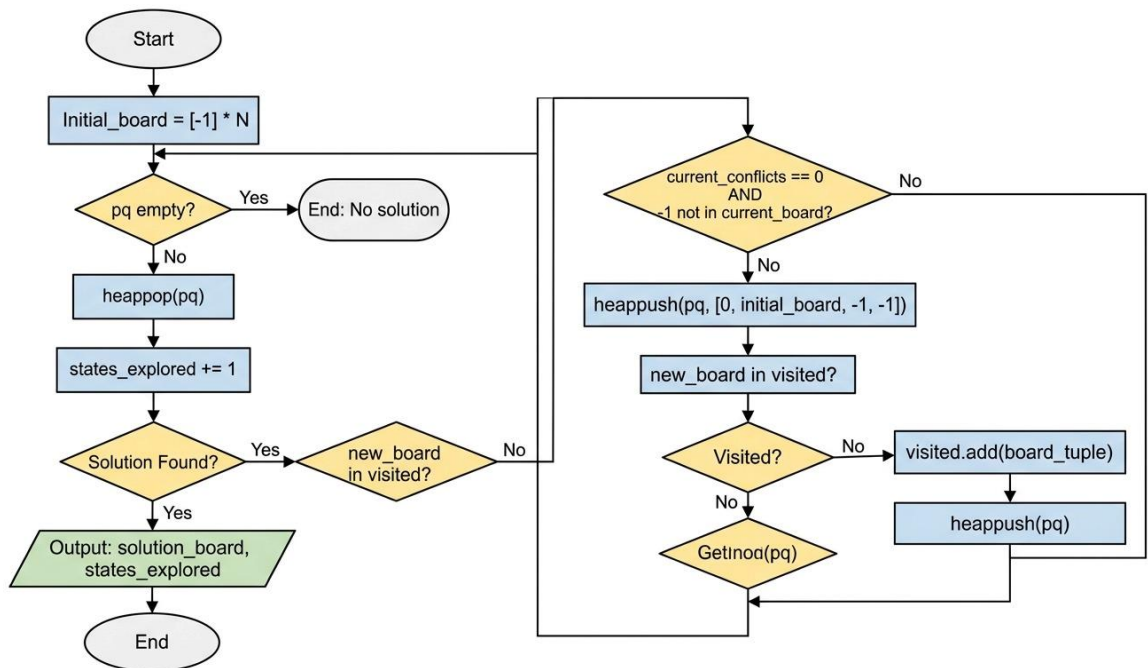
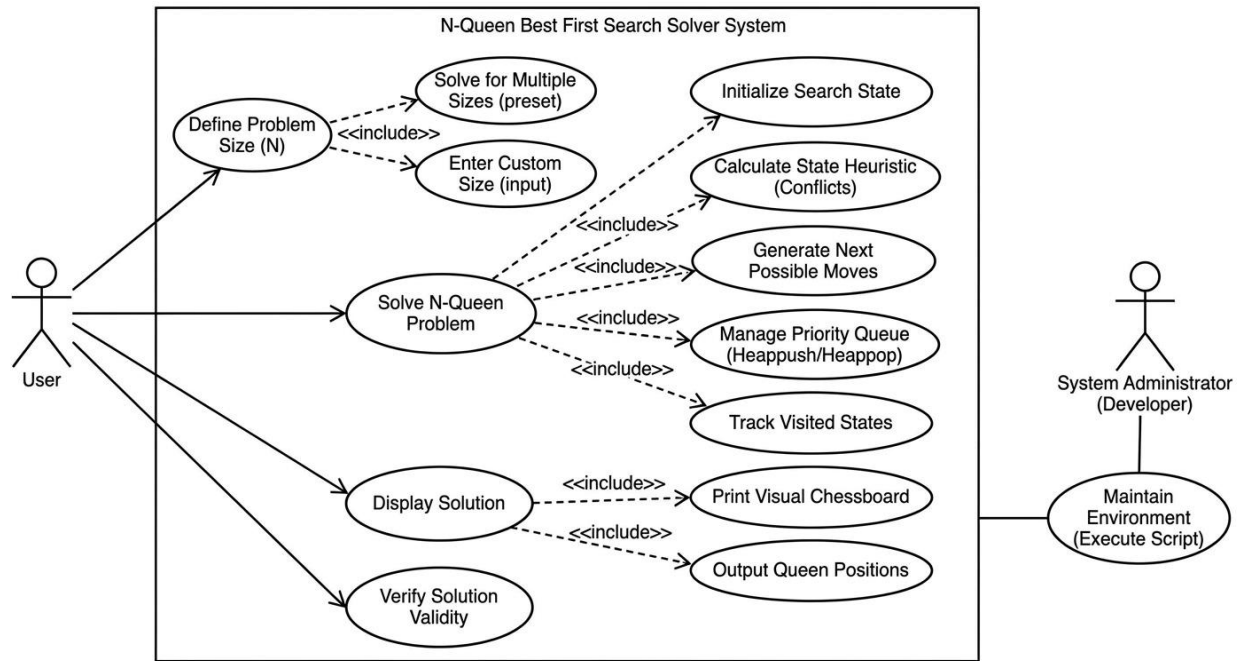
Graph BFS can get stuck in cycles if costs are not strictly positive/consistent, and does not guarantee shortest path.

N-Queens BFS is not guaranteed to find a solution (can get trapped in local minima if all neighbours increase conflicts), but with a suitable tie-breaking and random restart it works extremely well in practice.

Comparison of Time and Space Complexity “summary of the results”:

	First Code (Backtracking 2D)	Second Code (1D Representation)
Time Complexity	$O(k \times n)O(k \times \log n)$	$O(G \times P \times n^2)$
Space Complexity	$O(n^2)$ (full frontier list + sorting)	$O(n)$ (heap + few variables)
Efficiency Focus	Simpler code but higher memory footprint	Memory-optimized, faster extraction $O(\log n)$
Optimization Result	Higher memory consumption, only for small n	Significant space reduction + faster node selection

Diagrams:



Genetic Algorithm

The main idea of Genetic Algorithm:

A Genetic Algorithm is a search and optimization technique inspired by natural evolution.

It tries to solve problems the same way nature evolves living organisms: by selection, reproduction, and mutation.

Instead of solving a problem directly, it:

- Starts with many random solutions
- Keeps improving them over time
- Eventually finds a very good (or optimal) solution

Think of it like this:

You have a population of solutions. The best ones survive and reproduce, while the weak ones disappear. Over generations, solutions become stronger and better.

The N-Queen problem solver using Genetic Search Algorithm implemented code :

```
import random

import time

from utils import generate_random_board, calculate_conflicts

# Fitness function

def fitness(board):

    return calculate_conflicts(board) # LOWER is better

# Grade population (average fitness)

def grade(pop):

    total = sum(fitness(x) for x in pop)

    return total / len(pop)

# Create population

def population(count, n):

    return [generate_random_board(n) for _ in range(count)]
```

```

# Evolve function (same style as your first code)

def evolve(pop, retain=0.2, random_select=0.05, mutate=0.1):

    # 1. Sort population (best first)

    graded = [(fitness(x), x) for x in pop]

    graded = [x[1] for x in sorted(graded, key=lambda x: x[0])]

    # Select best

    retain_length = int(len(graded) * retain)

    parents = graded[:retain_length]

    # Randomly add others (diversity)

    for individual in graded[retain_length:]:

        if random.random() < random_select:

            parents.append(individual)

    # Mutation

    for individual in parents:

        if random.random() < mutate:

            pos = random.randint(0, len(individual) - 1)

            individual[pos] = random.randint(0, len(individual) - 1)

    # Crossover

    parents_length = len(parents)

    desired_length = len(pop) - parents_length

    children = []

    while len(children) < desired_length:

        male = random.choice(parents)

        female = random.choice(parents)

        if male != female:

            half = len(male) // 2

            child = male[:half] + female[half:]

```

```

        children.append(child)
    parents.extend(children)
    return parents

def solve(n):
    start_time = time.time()

    pop_size = 50
    max_generations = 500
    pop = population(pop_size, n)
    best_solution = None

    for generation in range(max_generations):
        pop = evolve(pop)

        # sort to get best
        pop = sorted(pop, key=lambda x: fitness(x))

        best = pop[0]

        if fitness(best) == 0:
            best_solution = best
            break

    end_time = time.time()
    return best_solution, {
        "time": end_time - start_time,
        "success": best_solution is not None
    }

solution, info = solve(8)
print("Solution:", solution)
print("Info:", info)

```

Summary of the Code Technique:

The code solves the N-Queens problem using a Genetic Algorithm with a compact 1-D board representation. Instead of exploring the search space systematically using backtracking, the algorithm evolves a population of candidate solutions through selection, crossover, mutation, and population replacement until a valid board configuration is found.

The approach is population-based and probabilistic, allowing the algorithm to search multiple candidate solutions simultaneously while gradually improving solution quality over generations.

Technique in the code:

Board representation:

The board is represented as a 1-D list of length n , where the index represents the row number and the value stored at that index represents the column position of the queen in that row. This guarantees one queen per row automatically.

Fitness function (fitness(board)):

The fitness function calculates the number of queen conflicts using `calculate_conflicts(board)`. Lower fitness values are better because fewer conflicts indicate a board closer to a valid solution.

Population initialization (population(count, n)):

The algorithm starts by generating a random population of candidate boards using `generate_random_board(n)`.

Population grading (grade(pop)):

The average fitness of the population is computed to evaluate the overall quality of the generation.

Evolution process (evolve(pop)):

The `evolve` function performs the main Genetic Algorithm operations:

Selection:

The population is sorted based on fitness values, and the best individuals are retained as parents.

Diversity preservation:

Some weaker individuals are randomly selected and added to the parent pool to maintain genetic diversity and reduce premature convergence.

Mutation:

Some parents are randomly mutated by changing the position of one queen. This introduces randomness and helps explore new areas of the search space.

Crossover:

Two parents are randomly selected, and a child is produced by combining the first half of one parent with the second half of the other parent.

Main search loop (solve(n)):

The solve function repeatedly evolves the population over multiple generations. After each generation, the best board is selected and checked. If the board has zero conflicts, the algorithm terminates successfully and returns the solution together with execution metrics.

The algorithm performs stochastic optimization rather than deterministic search, making it suitable for exploring large search spaces where exhaustive methods become expensive.

Analysis of the code:

Time Complexity:

The algorithm operates on a population of candidate solutions and evolves them over multiple generations using selection, crossover, and mutation.

In each generation, the algorithm first evaluates the fitness of all individuals in the population by counting the number of queen conflicts. This takes $O(n^2)$ time per individual due to pairwise conflict checking between queens.

Since there are P individuals in the population, the total fitness evaluation cost per generation is: $O(P \times n^2)$

After evaluation, the population is sorted based on fitness, which requires: $O(P \log P)$

Then selection, crossover, and mutation are applied to generate a new population. These operations require scanning and modifying individuals of size n , giving: $O(P \times n)$

per generation.

Therefore, the algorithm repeats this process for G generations, leading to a total worst-case time complexity of: $O(G \times (P \times n^2 + P \log P + P \times n))$

Since the dominant term is $P \times n^2$, the final simplified complexity becomes: $O(G \times P \times n^2)$

Space Complexity:

The algorithm maintains a population of candidate solutions where each solution (board) is represented as a 1-D array of size n , storing the column position of the queen in each row.

Therefore, storing a single board requires: $O(n)$

Since the algorithm keeps a population of P individuals, the total space used by the population is: $O(P \times n)$

During evolution, additional temporary lists are created for:

selection

crossover

mutation

sorting

However, these structures do not exceed the size of the population and are released after each generation, so they do not increase asymptotic space complexity.

Thus, the total space complexity of the algorithm is: $O(P \times n)$

The N-Queen problem solver using Genetic Search Algorithm implemented code:

```
import time

import random

from utils import generate_random_board, calculate_conflicts

def solve(n):

    start_time = time.time()

    #initialize the parameters

    population_size = 50

    mutation_rate = 0.2

    max_generations = 500

    #helper functions

    def fitness(board):

        return -calculate_conflicts(board) # higher is better
```

```

def selection(population):
    population.sort(key=lambda x: fitness(x), reverse=True)
    return population[:10] # top 10

def crossover(parent1, parent2):
    n = len(parent1)
    point = random.randint(0, n - 1)
    return parent1[:point] + parent2[point:]

def mutate(board):
    if random.random() < mutation_rate:
        row = random.randint(0, n - 1)
        col = random.randint(0, n - 1)
        board[row] = col
    return board

#initial population
population = [generate_random_board(n) for _ in range(population_size)]

best_solution = None

steps = 0

#evolution loop
for generation in range(max_generations):
    steps += 1

    #sort by fitness
    population.sort(key=lambda x: fitness(x), reverse=True)

    best = population[0]

    #check solution
    if calculate_conflicts(best) == 0:
        best_solution = best
        break

```

```

#selection
selected = selection(population)

#create next generation
new_population = []

while len(new_population) < population_size:
    parent1 = random.choice(selected)
    parent2 = random.choice(selected)
    child = crossover(parent1, parent2)
    child = mutate(child)
    new_population.append(child)

population = new_population

end_time = time.time()

metrics = {
    "algorithm": "Genetic Algorithm",
    "time": end_time - start_time,
    "steps": steps,
    "success": best_solution is not None
}

return best_solution, metrics

```

The summary of code:

What it does: It tries to find a board configuration where no queens attack each other “Where conflicts are equal 0”.

Main Steps

1. Initialization

Sets parameters:

Population size = 50

Mutation rate = 0.2

Max generations = 500

Generates random boards using “generate_random_board”

2. Fitness Function

fitness(board):

Returns negative number of conflicts

Fewer conflicts → higher fitness (better solution)

3. Selection

Sorts population by fitness

Selects the **top 10 individuals** (best boards)

4. Crossover

Combines two parents:

Picks a random split point

Creates a child from parts of both parents

5. Mutation

With 20% probability:

Randomly changes a queen's position

6. Evolution Loop

Repeats for up to 500 generations:

- 1.Sort population by fitness
- 2.Check if best solution has 0 conflicts
- 3.Select best individuals
- 4.Generate new population using crossover + mutation

7. Output

Returns:

best_solution (if found)

metrics:

Execution time

Number of steps (generations)

Success status (True/False)

Code Analysis:

1. Structure:

The code is well-organized: Main function: solve(n)

Helper functions:

- fitness
- selection
- crossover
- mutate

This makes the code **modular, readable, and easy to modify.**

2. Work-flow:

- 1-Start timing
- 2- Generate an initial random population
- 3-Run the evolution loop
- 4- In each generation:
 - Sort population by fitness
 - Select the best individuals
 - Apply crossover and mutation
 - Create a new population
- 5- Stop if a solution is found or max generations reached
- 6- Return the solution and performance metrics

3. Key Observations:

Strengths :

- Clear and simple fitness function
- Proper termination condition (0 conflicts)

- Useful metrics (time, steps, success)
- Mutation is probabilistic (adds randomness)
- Good use of randomness for exploration

Weaknesses :

- **Selection is limited** : Only top 10 are chosen → reduces diversity
- **Simple crossover** : Single-point crossover may not produce strong offspring
- **Weak mutation** : Only modifies one queen → limited impact
- **No elitism** : Best solution might be lost in next generation
- **Repeated conflict calculation** : Could be costly for large

4. Performance Behavior

Works efficiently for small n

For large n:

- May require more generations
- May fail to find a solution

5. Code Quality

- Clean and readable
- Well-structured
- Easy to understand

Improvements:

- Add more comments

The Differences between Two Codes:

1. Fitness Function Representation

First code:

```
return calculate_conflicts(board)
```

Lower value = better solution.

Second code:

return -calculate_conflicts(board)

Higher value = better solution.

2. Code Structure

First code:

Uses a **modular structure**: fitness, grade, population & evolve.

More reusable and scalable design.

Second code:

All logic is inside solve().

More compact but less reusable.

3. Selection Strategy

First code:

retain percentage + random selection

Keeps diversity and avoids premature convergence.

Second code:

top 10 individuals only

Stronger elitism, but less diversity.

4. Crossover Method

First code:

Midpoint crossover:

half split

Second code:

Single-point random crossover:

random split point

First is more stable, second is more random.

5. Mutation Strategy

First code:

Mutation applied during **evolve()** on parents

Second code:

→ Mutation applied after **offspring creation**

→ Different timing of exploration:

First:

→ earlier exploration

Second:

→ later exploration

6. Population Evolution Style

First code:

→ Keeps part of parents + adds children (structured evolution)

Second code:

→ Completely replaces population each generation

7. Metrics and Tracking

First code:

Returns: time & success only

Second code:

Returns: time, steps, success flag & algorithm name

Second code provides richer performance analysis.

8. Diversity Handling

First code:

Explicit diversity via: random selection + retain

Second code: No explicit diversity mechanism.

Summary:

Both codes implement a **Genetic Algorithm for the N-Queens problem**, using a 1-D board representation and evolutionary operations (selection, crossover, mutation).

However:

→ The first implementation is **more modular and diversity-preserving**

→ The second implementation is **simpler and more elitist**

Despite structural differences, both rely on the same core idea:

→ Iteratively improve a population of candidate solutions until a configuration with zero conflicts is found.

Time Complexity (Both Codes):

→ Both algorithms perform evolutionary search over a population across multiple generations.

Let:

→ **P** = population size

→ **n** = board size

→ **G** = number of generations

Each generation includes:

→ fitness evaluation → $O(n^2)$

→ sorting → $O(P \log P)$

→ crossover + mutation → $O(n)$

So total time complexity is:

→ $O(n^2)$

Since **n^2 dominates $\log P$** , the final simplified form is:

→ $O(n^2)$

Space Complexity (Both Codes):

→ Each individual is a 1-D array of size n , and there are P individuals in the population.

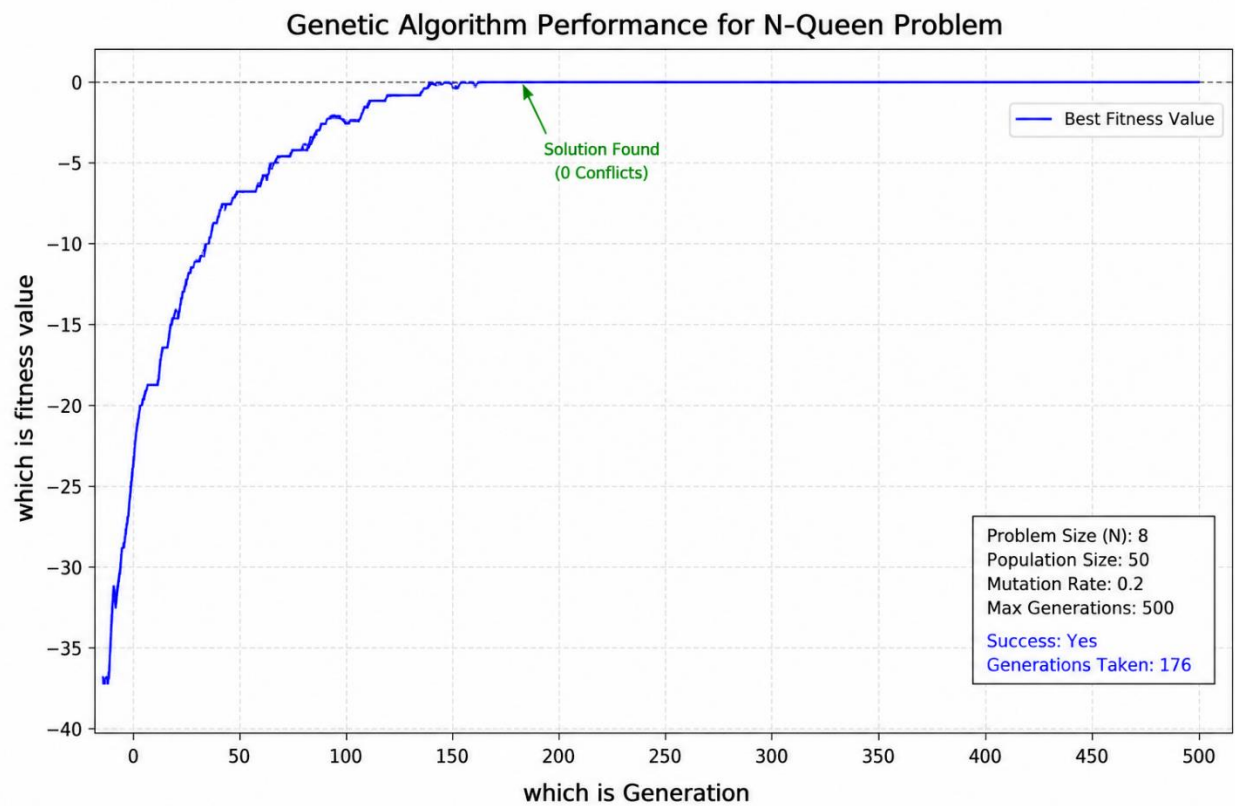
So memory usage is:

→ $O(n)$

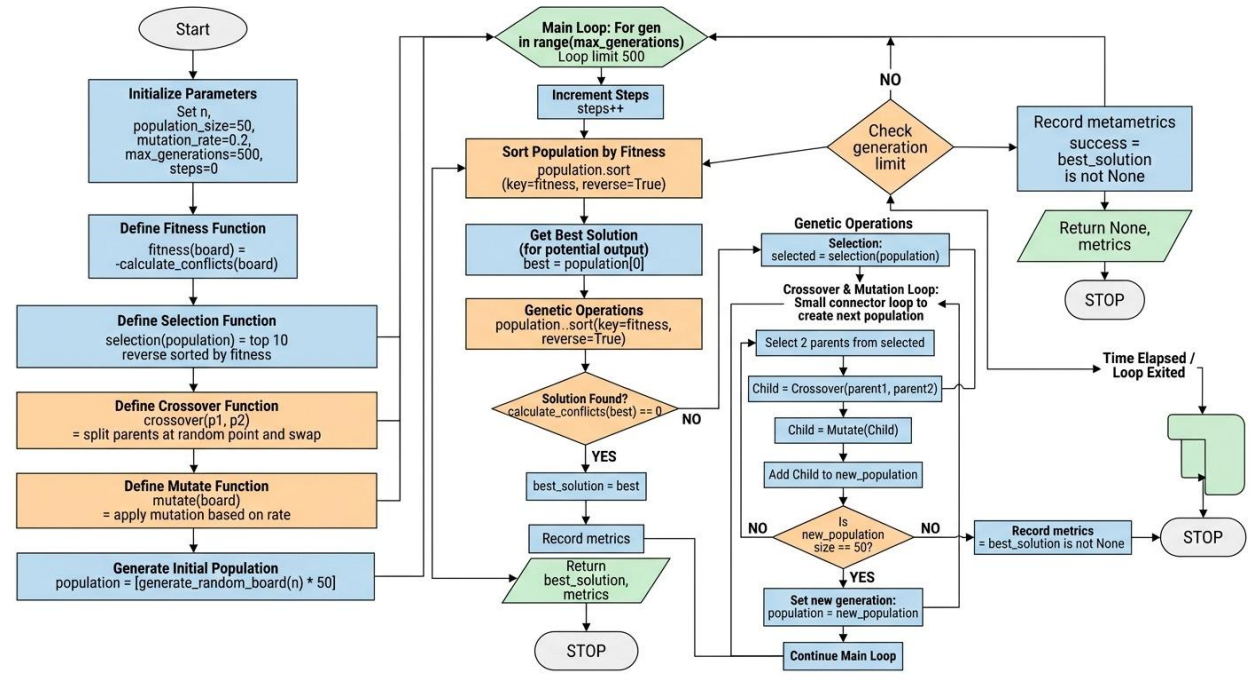
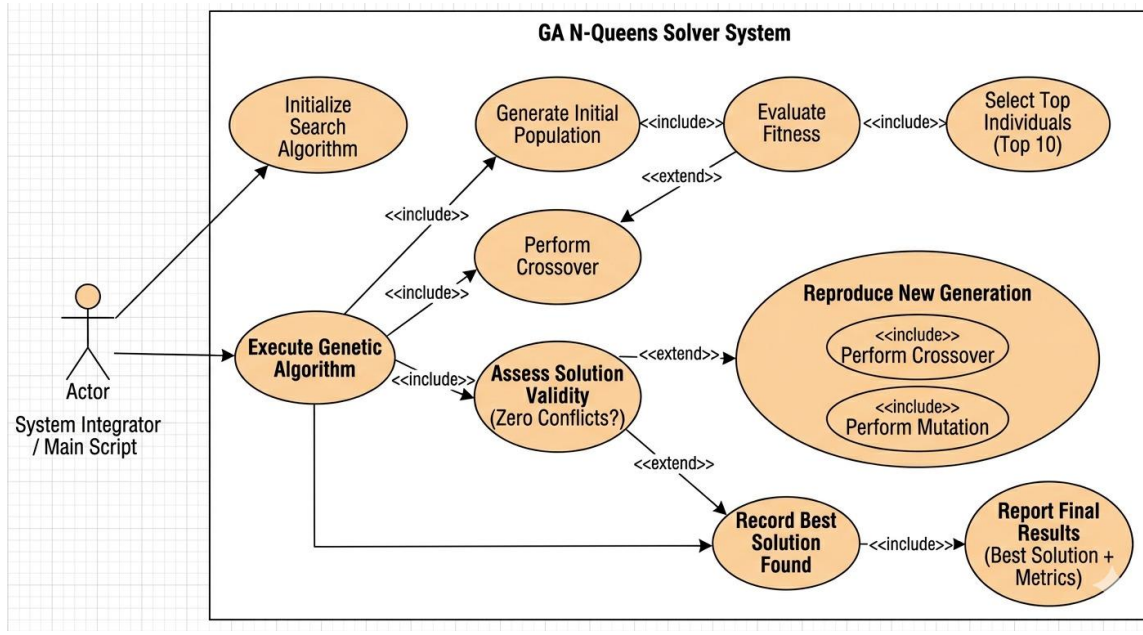
→ Temporary arrays used in selection and crossover do not exceed this bound

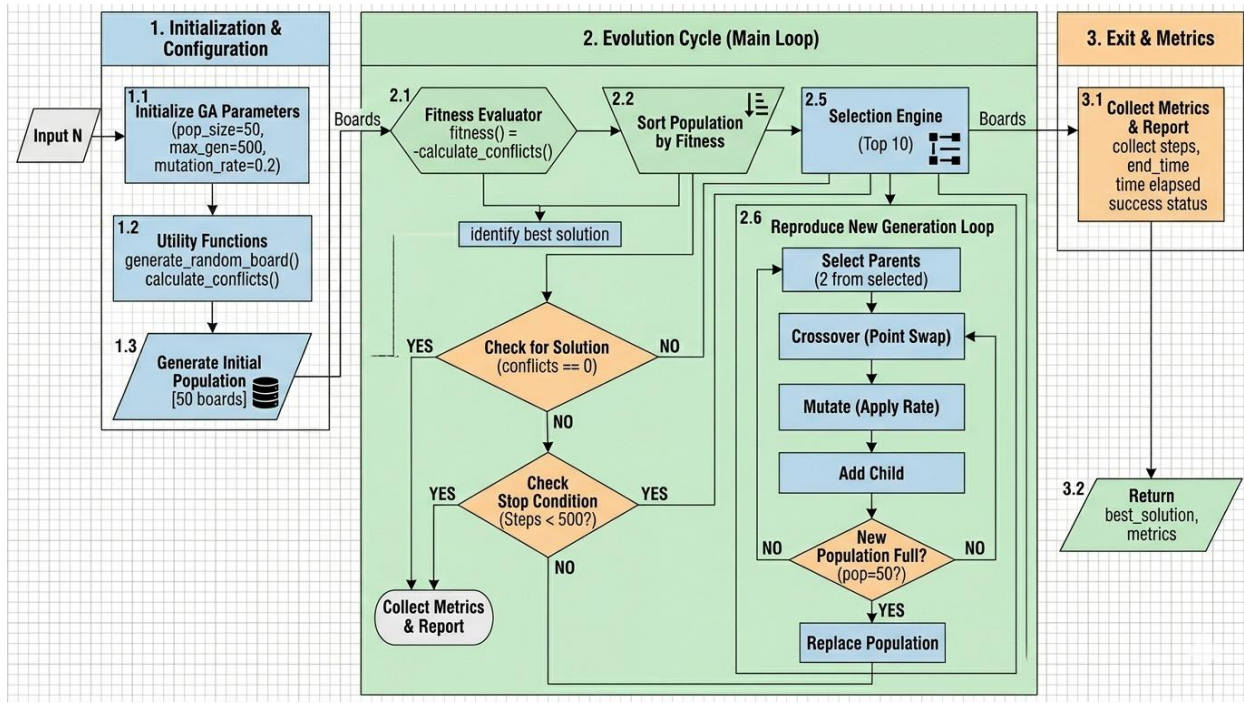
Comparison of Time and Space Complexity “summary of the results”:

	First Code (Backtracking 2D)	Second Code (1D Representation)
Time Complexity	$O(n! * n)$	$O(n^2)$
Space Complexity	$O(n^2)$	$O(n)$
Efficiency Focus	Exhaustive search with pruning (DFS Backtracking)	Probabilistic search using population-based evolution
Optimization Result	High memory usage, but systematic and complete search	Better memory efficiency, faster exploration of large search space



Diagrams:





Display File

```
import tkinter as tk
from tkinter import scrolledtext, messagebox
import threading

# Import the algorithm functions
from algorithms.backtracking import solve as backtracking
from algorithms.hill_climbing import solve as hill_climbing
from algorithms.best_first import solve as best_first
from algorithms.genetic import solve as genetic

# Helper functions to format output (same as before, but return strings)
def format_board(solution, n):
    if not solution:
        return "No solution found.\n"
    lines = []
    for row in solution:
        line = ['.'] * n
        line[row] = 'Q'
        lines.append(' '.join(line))
    return '\n'.join(lines)

def format_metrics(metrics):
    return (f"Time: {metrics.get('time', 0):.6f} sec\n"
            f"Steps: {metrics.get('steps', 'N/A')}\n"
            f"Success: {metrics.get('success', False)}")

def format_comparison(results):
    header = f"{Algorithm':<20}{Time (s)':<15}{Steps':<15}{Success':<10}\n"
    header += "-" * 60 + "\n"
    rows = []
    for m in results:
        rows.append(f"{m.get('algorithm', 'Unknown'):<20}"
                    f"{m.get('time', 0):<15.6f}"
                    f"{m.get('steps', 'N/A'):<15}"
                    f"{m.get('success', False)}")
    return header + '\n'.join(rows)
```

```

# GUI functions
def run_gui():
    """Create the main window and start the Tkinter event loop."""
    root = tk.Tk()
    root.title("N-Queens Solver")
    root.geometry("700x600")

    # Input frame
    input_frame = tk.Frame(root)
    input_frame.pack(pady=10)

    tk.Label(input_frame, text="Enter N:").pack(side=tk.LEFT, padx=5)
    n_entry = tk.Entry(input_frame, width=5)
    n_entry.pack(side=tk.LEFT, padx=5)

    # Output area
    output_text = scrolledtext.ScrolledText(root, width=80, height=30, font=("Courier", 10))
    output_text.pack(pady=10, padx=10, fill=tk.BOTH, expand=True)

    # Function to run the algorithms in a separate thread
    def solve_thread(n):
        results = []
        algorithms = [
            ("Backtracking Search", backtracking),
            ("Hill Climbing Search", hill_climbing),
            ("Best First Search", best_first),
            ("Genetic Search", genetic)
        ]

        for name, algo in algorithms:
            output_text.insert(tk.END, f"\n{' '*50}\nAlgorithm: {name}\n{' '*50}\n")
            sol, met = algo(n)
            output_text.insert(tk.END, "\nBoard:\n")
            output_text.insert(tk.END, format_board(sol, n))
            output_text.insert(tk.END, "\nMetrics:\n")
            output_text.insert(tk.END, format_metrics(met))
            results.append(met)

        output_text.insert(tk.END, "\n" + "="*50 + "\n")
        output_text.insert(tk.END, "COMPARISON TABLE\n")

```

```

output_text.insert(tk.END, "="*50 + "\n")
output_text.insert(tk.END, format_comparison(results))

# Re-enable the solve button
solve_button.config(state=tk.NORMAL)

# Function called when the Solve button is clicked
def on_solve_click():
    try:
        n = int(n_entry.get())
        if n <= 0:
            messagebox.showerror("Invalid N", "N must be greater than 0.")
            return
    except ValueError:
        messagebox.showerror("Invalid Input", "Please enter a valid integer.")
        return

    # Disable button, clear output, start thread
    solve_button.config(state=tk.DISABLED)
    output_text.delete(1.0, tk.END)
    threading.Thread(target=solve_thread, args=(n,), daemon=True).start()

solve_button = tk.Button(input_frame, text="Solve", command=on_solve_click)
solve_button.pack(side=tk.LEFT, padx=10)

root.mainloop()

```

→ It Contains The GUI functions and The printing functions that arrange and print the output inside the GUI and The threading functions to keep GUI alive during long searches and The error handling function.

→ it called from the main and it's functionality is to take input from user and handling any input errors then call the algorithms and send the input to them and when each algorithm file finish, they send the board coordinates and the metrics of the operation, it receive all of that from each algorithm and make the boards format "according to the received board coordinates" which display them to the user in a way to have the ability to make a comparison.

Similar applications in the market

- **Complete Solvers (like Backtracking Search algo):** Products like Gecode (C++), Choco (Java), and the commercial IBM ILOG CPLEX use systematic, complete search to guarantee finding a solution if one exists.
- **Incomplete Solvers (like Hill Climbing Search algo, Best First Search algo, Genetic Search algo):** Toolkits like Google OR-Tools and MiniZinc use heuristics and metaheuristics to find "good enough" solutions very quickly for large, complex problems where exhaustive search isn't feasible.

Development Platform

Programming language	Python 3
GUI framework	Tkinter (built-in, used for window, buttons, scrolled text)
Threading	threading module (to keep GUI alive during long searches)
Data structures	Lists, Tuples, Heap (for Best-first), random module
Algorithms	Custom implementations (no external ML/AI libraries)
Development environment	Any Python IDE or text editor (e.g., VS Code, PyCharm) – not specified but assumed.